

Project: Teoria delle code

processi di arrivo, servizio e attesa: dal modello M/M/1 alle reti di code

1. Obiettivi della dispensa

Questa dispensa introduce la teoria delle code come caso di studio per un corso di metodi computazionali per modelli stocastici.

Gli obiettivi sono sei:

1. formalizzare una coda come processo stocastico a eventi discreti e derivare le equazioni di bilancio;
2. risolvere il modello M/M/1 e interpretarne la distribuzione stazionaria;
3. derivare le formule di Little per il numero medio di clienti e il tempo medio di attesa;
4. studiare le varianti M/M/k e M/M/1/K e discutere il loro significato applicativo;
5. costruire una simulazione a eventi discreti e confrontarla con i risultati analitici;
6. introdurre le reti di code di Jackson come estensione naturale al caso di piu' stazioni in serie o in rete.

Dal punto di vista del corso, la teoria delle code e' particolarmente utile perche' introduce in modo molto concreto i processi di birth-death, la distribuzione stazionaria come autovettore di un generatore, e la simulazione a eventi discreti come metodo computazionale alternativo all'integrazione numerica delle ODE.

2. Motivazione: cosa e' una coda e dove la incontriamo

Una coda nasce ogni volta che una risorsa limitata deve servire richieste che arrivano in modo casuale.

Esempi immediati e concreti:

- **Cassa del supermercato.** I clienti arrivano a intervalli irregolari. Se la cassa e' occupata, si accodano. Quanto aspettano in media? Quando conviene aprire una seconda cassa?
- **Pronto soccorso.** I pazienti arrivano con urgenze diverse. Il medico impiega tempi variabili per ogni visita. Quanti pazienti si accumulano in sala d'attesa nelle ore di punta?
- **Server web.** Le richieste HTTP arrivano a raffiche. Il server impiega un tempo variabile per rispondere. Sotto quale carico il tempo di risposta diventa inaccettabile?
- **Call center.** Le chiamate arrivano a caso. Gli operatori hanno tempi di conversazione variabili. Quanti operatori servono per non tenere i clienti in attesa piu' di due minuti?
- **Semaforo.** Le auto arrivano a un incrocio. Il verde dura un tempo fisso. Quante auto si accumulano con una certa intensita' di traffico?
- **Scarico di un porto.** Le navi arrivano, aspettano una banchina libera, vengono scaricate. Con quante banchine si minimizza il tempo di attesa?

In tutti questi casi la struttura e' la stessa: arrivi casuali, servizio che richiede tempo, capacita' limitata. La teoria delle code fornisce strumenti matematici e computazionali per analizzare questi sistemi in modo preciso.

3. Notazione di Kendall

Per descrivere una coda in modo compatto si usa la notazione di Kendall:

$$A/S/k/K,$$

dove:

- A descrive il processo di arrivo;
- S descrive il processo di servizio;
- k e' il numero di server;
- K e' la capacita' massima del sistema (clienti in servizio piu' in attesa); se omissso si assume $K = \infty$.

I simboli piu' comuni per A e S sono:

- M (Markoviano): tempi interarrivo o di servizio distribuiti esponenzialmente;
- D (Deterministico): tempi fissi;
- G (Generale): distribuzione arbitraria.

Quindi:

- $M/M/1$: arrivi poissoniani, servizio esponenziale, un server, capacita' illimitata. E' il modello base.
- $M/M/k$: come sopra ma con k server paralleli. Modella il call center con piu' operatori.

- $M/M/1/K$: un server, ma al massimo K clienti nel sistema. Modella il parcheggio con K posti o il buffer di un router.
- $M/D/1$: arrivi poissoniani, servizio a tempo fisso. Modella il semaforo o una macchina industriale a cadenza regolare.

4. Il modello M/M/1

4.1 Arrivi poissoniani

Nel modello M/M/1, i clienti arrivano secondo un processo di Poisson con tasso λ .

Questo significa che:

- il numero di arrivi in un intervallo di lunghezza t e' una variabile di Poisson con media λt ;
- i tempi tra arrivi successivi sono distribuiti esponenzialmente con media $1/\lambda$;
- gli arrivi in intervalli disgiunti sono indipendenti.

Esempio concreto. In un bar, in media arrivano $\lambda = 4$ clienti al minuto durante l'ora di punta. Il tempo medio tra un cliente e il successivo e' $1/\lambda = 15$ secondi. Alcuni arrivano quasi contemporaneamente, altri distanziati di piu' di un minuto: questa variabilita' e' catturata esattamente dalla distribuzione esponenziale.

4.2 Servizio esponenziale

Il tempo di servizio di ogni cliente e' distribuito esponenzialmente con tasso μ , quindi con media $1/\mu$.

La proprieta' fondamentale della distribuzione esponenziale e' la mancanza di memoria: sapere che un cliente e' gia' stato servito per t secondi non cambia la distribuzione del tempo rimanente. Questo rende il modello trattabile analiticamente.

Esempio concreto. Il barista impiega in media $1/\mu = 20$ secondi per preparare un ordine. A volte e' un caffe' espresso (10 secondi), a volte un cappuccino con latte montato (45 secondi). La distribuzione esponenziale cattura questa variabilita'.

4.3 Stato del sistema e processo di birth-death

Lo stato del sistema al tempo t e' il numero di clienti presenti, in servizio piu' in attesa:

$$N(t) \in \{0, 1, 2, 3, \dots\}.$$

La dinamica di $N(t)$ e' un processo di birth-death:

- un **birth** (arrivo) avviene con tasso λ qualunque sia lo stato corrente;
- una **death** (partenza dopo servizio) avviene con tasso μ se $N(t) \geq 1$, e con tasso 0 se $N(t) = 0$.

Il generatore infinitesimale del processo ha la forma tridiagonale:

	0	1	2	3	...
0	$-\lambda$	λ	0	0	...
1	μ	$-(\lambda + \mu)$	λ	0	...
2	0	μ	$-(\lambda + \mu)$	λ	...
3	0	0	μ	$-(\lambda + \mu)$	...
$\vdots$					$\ddots$

4.4 Condizione di stabilita' e carico

Il parametro fondamentale del modello e' il **fattore di carico** (o traffico):

$$\rho = \frac{\lambda}{\mu}.$$

Interpretazione immediata: ρ e' la frazione di tempo che il server trascorre occupato. Se $\rho < 1$, il server riesce a smaltire i clienti in media piu' velocemente di quanto arrivino, e il sistema e' stabile. Se $\rho \geq 1$, la coda cresce senza limite nel tempo.

Esempio concreto. Nel bar dell'esempio, $\lambda = 4$ clienti/minuto e $\mu = 3$ clienti/minuto (il barista e' lento). Allora $\rho = 4/3 > 1$: la coda esplode. Se invece $\mu = 5$, allora $\rho = 0.8$: il sistema e' stabile, ma il server e' occupato l'80% del tempo.

Il punto critico $\rho \rightarrow 1^-$ e' una delle soglie piu' importanti della teoria: avvicinarsi alla saturazione fa crescere l'attesa in modo drammatico, come vedremo.

5. Distribuzione stazionaria di M/M/1

5.1 Equazioni di bilancio

In stazionarieta', per ogni stato n il flusso di probabilita' entrante deve uguagliare quello uscente. Per lo stato $n \geq 1$:

$$(\lambda + \mu)\pi_n = \lambda\pi_{n-1} + \mu\pi_{n+1}.$$

Per lo stato 0:

$$\lambda\pi_0 = \mu\pi_1.$$

Queste equazioni si risolvono per ricorrenza. Dalla condizione per $n = 0$:

$$\pi_1 = \frac{\lambda}{\mu}\pi_0 = \rho\pi_0.$$

Procedendo per induzione si ottiene:

$$\pi_n = \rho^n \pi_0.$$

5.2 Normalizzazione

La condizione $\sum_{n=0}^{\infty} \pi_n = 1$ fornisce:

$$\pi_0 \sum_{n=0}^{\infty} \rho^n = 1.$$

Per $\rho < 1$ la serie geometrica converge e si ottiene $\pi_0 = 1 - \rho$, quindi:

$$\boxed{\pi_n = (1 - \rho)\rho^n, \quad n = 0, 1, 2, \dots}$$

Questa e' la distribuzione geometrica. E' la distribuzione stazionaria del numero di clienti in un sistema M/M/1.

Interpretazione. La probabilita' di trovare n clienti nel sistema decresce geometricamente con n . Piu' grande e' ρ , piu' lenta e' la decrescita, piu' spesso si trovano molti clienti in coda.

5.3 Numero medio di clienti nel sistema

$$L = \sum_{n=0}^{\infty} n\pi_n = \sum_{n=0}^{\infty} n(1 - \rho)\rho^n = \frac{\rho}{1 - \rho}.$$

Esempio concreto. Con $\rho = 0.5$ (server occupato il 50% del tempo): $L = 1$ cliente in media. Con $\rho = 0.8$: $L = 4$. Con $\rho = 0.9$: $L = 9$. Con $\rho = 0.95$: $L = 19$. La crescita e' fortemente non lineare: avvicinarsi alla saturazione fa esplodere le code.

5.4 Numero medio di clienti in attesa

Il numero medio di clienti in coda (non in servizio) e':

$$L_q = L - (1 - \pi_0) = \frac{\rho^2}{1 - \rho}.$$

La quantita' $1 - \pi_0 = \rho$ e' la probabilita' che il server sia occupato, cioe' il numero medio di clienti in servizio.

6. Le formule di Little

Le formule di Little sono forse il risultato piu' importante e piu' generale della teoria delle code. Valgono per qualunque sistema stazionario, indipendentemente dalla distribuzione degli arrivi e dei tempi di servizio.

6.1 Enunciato

Sia L il numero medio di clienti nel sistema, λ il tasso di arrivo effettivo e W il tempo medio che un cliente trascorre nel sistema (attesa piu' servizio). Allora:

$$\boxed{L = \lambda W.}$$

Analogamente, se L_q e' il numero medio in coda e W_q il tempo medio di attesa prima del servizio:

$$L_q = \lambda W_q.$$

6.2 Significato intuitivo

La formula $L = \lambda W$ dice che il numero medio di clienti nel sistema e' uguale al tasso con cui entrano moltiplicato per il tempo che ognuno ci trascorre. E' un risultato di conservazione del flusso, analogo alla legge di continuita' in idraulica.

Esempio concreto. In un negozio, in media ci sono $L = 10$ clienti presenti. I clienti arrivano a $\lambda = 2$ al minuto. Quindi ogni cliente trascorre in media $W = L/\lambda = 5$ minuti nel negozio (attesa piu' tempo di servizio), senza bisogno di misurarlo direttamente.

6.3 Applicazione a M/M/1

Dal valore di L e dalle formule di Little si ricavano tutti i tempi medi:

$$W = \frac{L}{\lambda} = \frac{1}{\mu - \lambda},$$

$$W_q = W - \frac{1}{\mu} = \frac{\rho}{\mu - \lambda} = \frac{\lambda}{\mu(\mu - \lambda)}.$$

Esempio concreto. Nel bar con $\lambda = 4$ clienti/minuto e $\mu = 5$ clienti/minuto ($\rho = 0.8$):

- tempo medio nel sistema: $W = 1/(5 - 4) = 1$ minuto;
- tempo medio di attesa prima del servizio: $W_q = 0.8/1 = 0.8$ minuti = 48 secondi.

Se il tasso di arrivo sale a $\lambda = 4.5$ ($\rho = 0.9$): $W = 1/(5 - 4.5) = 2$ minuti. Aggiungere mezzo cliente al minuto raddoppia il tempo di attesa.

7. Il modello M/M/k

7.1 Struttura

Nel modello M/M/k ci sono k server identici, ciascuno con tasso di servizio μ . I clienti arrivano con tasso λ e si accodano in un'unica fila, servita dal primo server che si libera.

Esempio concreto. Un call center con $k = 5$ operatori. Le chiamate arrivano a $\lambda = 8$ al minuto, ogni operatore gestisce in media $1/\mu = 1$ minuto per chiamata. La coda si forma solo se tutti e 5 gli operatori sono occupati.

Il fattore di carico per server è:

$$\rho = \frac{\lambda}{k\mu}.$$

La condizione di stabilità è $\rho < 1$, cioè $\lambda < k\mu$.

7.2 Distribuzione stazionaria

La distribuzione stazionaria ha la forma:

$$\pi_n = \begin{cases} \pi_0 \frac{(k\rho)^n}{n!} & \text{per } 0 \leq n \leq k, \\ \pi_0 \frac{k^k \rho^n}{k!} & \text{per } n > k, \end{cases}$$

dove π_0 si trova dalla condizione di normalizzazione.

7.3 Formula di Erlang C

La probabilita' che un cliente in arrivo trovi tutti i server occupati e debba aspettare e' data dalla formula di Erlang C:

$$C(k, \rho) = \frac{\frac{(k\rho)^k}{k!(1-\rho)}}{\sum_{n=0}^{k-1} \frac{(k\rho)^n}{n!} + \frac{(k\rho)^k}{k!(1-\rho)}}.$$

Il tempo medio di attesa e':

$$W_q = \frac{C(k, \rho)}{k\mu - \lambda}.$$

Esempio concreto. Call center con $k = 5$ operatori, $\lambda = 3$ chiamate/min, $\mu = 1$ chiamata/min per operatore. Il carico e' $\rho = 3/5 = 0.6$. La formula di Erlang C da' la probabilita' di attesa e il tempo medio in coda, permettendo di calcolare quanti operatori servono per garantire un'attesa media inferiore a 30 secondi.

8. Il modello M/M/1/K

8.1 Struttura e motivazione

Nel modello M/M/1/K la capacita' del sistema e' limitata a K clienti. Un cliente che arriva quando il sistema e' pieno viene rifiutato (o abbandona).

Esempi concreti:

- un parcheggio con K posti: se e' pieno, le auto non entrano;
- un buffer di rete con K pacchetti: se e' pieno, i pacchetti in eccesso vengono scartati;
- una sala d'attesa con K sedie: i clienti in piedi lasciano il negozio;
- un pronto soccorso che reindirizza pazienti all'ospedale vicino quando supera la capienza.

8.2 Distribuzione stazionaria

Con capacita' finita il sistema e' sempre stabile (anche per $\rho \geq 1$). La distribuzione stazionaria e':

$$\pi_n = \begin{cases} \frac{1-\rho}{1-\rho^{K+1}} \rho^n & \text{per } \rho \neq 1, \\ \frac{1}{K+1} & \text{per } \rho = 1. \end{cases}$$

8.3 Probabilità di blocco e tasso effettivo

La probabilità di blocco, cioè la probabilità che un cliente in arrivo trovi il sistema pieno e venga rifiutato, è:

$$P_B = \pi_K = \frac{(1 - \rho)\rho^K}{1 - \rho^{K+1}}.$$

Il tasso effettivo di arrivi accettati è:

$$\lambda_{\text{eff}} = \lambda(1 - P_B).$$

Esempio concreto. Un parcheggio con $K = 20$ posti. Arrivano $\lambda = 10$ auto/ora, ogni auto resta in media $1/\mu = 2$ ore. Il carico è $\rho = \lambda/\mu = 20$. Calcolare P_B permette di stimare quante auto vengono rifiutate e se conviene ampliare il parcheggio.

9. Confronto tra modelli: il costo della saturazione

Uno degli aspetti più istruttivi della teoria è osservare come cambia il tempo di attesa avvicinandosi alla saturazione, per diversi modelli.

Per M/M/1:

$$W_q = \frac{\rho}{\mu(1 - \rho)}.$$

Per M/M/k a parità di carico totale $\lambda/\mu = k\rho$, il tempo di attesa è molto minore grazie alla riduzione della variabilità effettiva.

Il messaggio qualitativo è molto importante: non è la media del carico a determinare l'attesa, ma la combinazione di carico e variabilità. Un sistema vicino alla saturazione con alta variabilità nei tempi di servizio produce code molto più lunghe di uno con la stessa saturazione ma servizio regolare.

Esempio concreto. Un medico di base con tempi di visita molto variabili (alcuni pazienti richiedono 5 minuti, altri 45) produce code molto più lunghe di un sistema automatizzato con tempo di servizio fisso, anche a parità di numero medio di pazienti e di durata media della visita.

10. Simulazione a eventi discreti

10.1 Idea generale

La simulazione a eventi discreti (DES, Discrete Event Simulation) è un metodo molto generale per simulare sistemi stocastici. Invece di avanzare il tempo in passi fissi Δt , si avanza direttamente al prossimo evento rilevante.

Per una coda, gli eventi sono di due tipi:

- **arrivo:** un nuovo cliente entra nel sistema;
- **partenza:** un cliente termina il servizio e lascia il sistema.

La simulazione mantiene una lista di eventi futuri ordinata per tempo. A ogni passo:

1. si estrae l'evento con il tempo piu' piccolo;
2. si aggiorna lo stato del sistema;
3. si generano i nuovi eventi conseguenti.

Questo approccio e' molto piu' efficiente dell'avanzamento a passo fisso quando gli eventi sono rari o molto distanziati nel tempo.

10.2 Struttura della simulazione per M/M/1

Lo stato del sistema e' il numero corrente di clienti n .

All'inizializzazione:

- si genera il primo arrivo al tempo $t_1 \sim \text{Exp}(\lambda)$;
- la lista degli eventi contiene solo questo arrivo.

Quando si processa un **arrivo** al tempo t :

- si incrementa n ;
- si genera il prossimo arrivo al tempo $t + \text{Exp}(\lambda)$;
- se $n = 1$ (il sistema era vuoto), si genera la partenza al tempo $t + \text{Exp}(\mu)$.

Quando si processa una **partenza** al tempo t :

- si decrementa n ;
- se $n \geq 1$ (ci sono ancora clienti), si genera la prossima partenza al tempo $t + \text{Exp}(\mu)$.

10.3 Perche' la simulazione e' importante

La simulazione a eventi discreti e' importante per tre ragioni.

Primo, permette di verificare i risultati analitici: se la simulazione e l'analisi concordano, si ha fiducia in entrambi.

Secondo, permette di estendere il modello oltre i casi analiticamente trattabili: tempi di servizio non esponenziali, priorit  tra clienti, server con guasti, arrivi a raffiche, reti di code complesse.

Terzo, e' un esempio molto pulito di simulazione di un processo stocastico a eventi discreti, con una struttura che si generalizza a molti altri problemi del corso.

11. Reti di code di Jackson

11.1 Motivazione

In molte applicazioni reali, i clienti non attraversano una sola coda ma una sequenza di stazioni.

Esempi concreti:

- in un ospedale, un paziente passa per il triage, poi la visita, poi la radiologia, poi la farmacia;
- in una fabbrica, un pezzo attraversa tornio, fresatrice, assemblaggio, collaudo;
- in un call center con specializzazioni, una chiamata puo' essere trasferita da un operatore a un altro;
- in una rete di computer, un pacchetto attraversa piu' router prima di arrivare a destinazione.

11.2 Il teorema di Jackson

Per una rete di J stazioni con:

- arrivi esterni poissoniani a ogni stazione j con tasso γ_j ;
- routing probabilistico: un cliente che completa il servizio alla stazione j si sposta alla stazione i con probabilita' p_{ji} , oppure lascia la rete con probabilita' $1 - \sum_i p_{ji}$;
- servizio esponenziale alla stazione j con tasso μ_j ;

il teorema di Jackson afferma che, in stazionarieta', la distribuzione congiunta del numero di clienti nelle J stazioni e' il prodotto delle distribuzioni marginali:

$$\pi(n_1, n_2, \dots, n_J) = \prod_{j=1}^J \pi_j(n_j),$$

dove ogni π_j e' la distribuzione di una M/M/1 con tasso di arrivo λ_j , il tasso totale che arriva alla stazione j (da fuori e dagli altri nodi).

Questo risultato, noto come **product-form solution**, e' notevole: nonostante le stazioni siano correlate (l'output di una e' l'input di un'altra), la distribuzione congiunta fattorizza come se fossero indipendenti.

11.3 Equazioni di traffico

I tassi λ_j si trovano risolvendo le **equazioni di traffico** (o equazioni di bilanciamento del flusso):

$$\lambda_j = \gamma_j + \sum_{i=1}^J \lambda_i p_{ij}, \quad j = 1, \dots, J.$$

Questo e' un sistema lineare in λ_j , risolvibile con metodi elementari.

Esempio concreto. In una rete con due stazioni, i pazienti arrivano alla stazione 1 (triage) con tasso $\gamma_1 = 5/\text{ora}$. Il 60% viene rimandato a casa, il 40% passa alla stazione 2 (visita). Nessuno arriva direttamente alla stazione 2 dall'esterno. Le equazioni di traffico danno $\lambda_1 = 5$, $\lambda_2 = 0.4 \times 5 = 2$ pazienti/ora. Poi si applica la formula M/M/1 a ciascuna stazione separatamente.

12. Domande scientifiche che il modello permette di studiare

Il modello e' utile per affrontare domande molto concrete.

1. Quanti server servono per garantire che la probabilita' di attesa sia inferiore al 10%?
2. Come cambia il tempo di attesa medio avvicinandosi alla saturazione?
3. Qual e' la probabilita' di blocco in un sistema M/M/1/K al variare di K ?
4. Quanto conta la variabilita' nei tempi di servizio rispetto alla media?
5. Quando conviene avere una sola coda lunga con piu' server invece di code separate?
6. In una rete di code, quale stazione e' il collo di bottiglia?

13. Schema del laboratorio

13.1 Laboratorio 1 - M/M/1: analisi e simulazione

Obiettivo

Verificare la distribuzione stazionaria e le formule di Little con una simulazione a eventi discreti.

Attivita'

1. fissare λ e μ con $\rho < 1$;
2. implementare la simulazione DES per M/M/1;
3. raccogliere la distribuzione empirica di $N(t)$ dopo il transitorio;
4. confrontare con la distribuzione geometrica teorica;
5. stimare L , W , L_q , W_q e confrontare con le formule analitiche.

Domande guida

- la distribuzione empirica converge a quella geometrica al crescere del tempo di simulazione?
- le formule di Little sono verificate numericamente?
- come cambia il tempo di convergenza al variare di ρ ?

Output richiesto

- codice sorgente;
- istogramma empirico vs distribuzione geometrica;
- tabella di L , W , L_q , W_q simulati vs analitici.

13.2 Laboratorio 2 - Il costo della saturazione

Obiettivo

Visualizzare in modo molto diretto l'esplosione del tempo di attesa avvicinandosi a $\rho = 1$.

Attività

1. fissare μ e variare λ su una griglia da 0.1μ a 0.99μ ;
2. calcolare W_q analitico e simulato per ogni valore di ρ ;
3. costruire il grafico di W_q in funzione di ρ ;
4. ripetere per M/M/2 con lo stesso carico totale λ/μ .

Domande guida

- a quale valore di ρ il tempo di attesa raddoppia rispetto al valore a $\rho = 0.5$?
- quanto M/M/2 riduce l'attesa rispetto a M/M/1 a parità di carico?
- la curva di W_q è convessa o concava in ρ ?

Output richiesto

- grafico di $W_q(\rho)$ per M/M/1 e M/M/2;
- commento quantitativo sull'effetto della saturazione.

13.3 Laboratorio 3 - M/M/1/K e probabilità di blocco

Obiettivo

Studiare come la capacità limitata modifica la distribuzione stazionaria e il tasso di clienti persi.

Attività

1. fissare λ , μ con $\rho > 1$ e variare K ;
2. calcolare analiticamente P_B e λ_{eff} per ogni K ;

3. simulare M/M/1/K e confrontare la distribuzione empirica con quella analitica;
4. costruire il grafico di P_B in funzione di K per diversi valori di ρ .

Domande guida

- esiste un valore di K oltre il quale aumentare la capacita' non riduce significativamente P_B ?
- per $\rho < 1$, la capacita' finita e' rilevante?
- come si comporta il sistema nel caso limite $\rho = 1$?

Output richiesto

- grafico di $P_B(K)$;
- confronto tra distribuzione simulata e analitica;
- commento applicativo.

13.4 Laboratorio 4 - Rete di code semplice

Obiettivo

Simulare una rete a due stazioni in serie e verificare il teorema di Jackson.

Attivita'

1. costruire una rete con due stazioni M/M/1 in tandem: l'output della prima e' l'input della seconda;
2. risolvere le equazioni di traffico;
3. simulare la rete e raccogliere le distribuzioni marginali di N_1 e N_2 ;
4. confrontare con le distribuzioni M/M/1 attese.

Domande guida

- le distribuzioni marginali sono geometriche come previsto dal teorema di Jackson?
- quale stazione e' il collo di bottiglia?
- cosa succede se si aggiunge un secondo server alla stazione satura?

Output richiesto

- schema della rete;
- istogrammi empirici per le due stazioni;
- discussione del collo di bottiglia.

14. Una possibile estensione teorica

14.1 Distribuzione del tempo di attesa

Oltre al numero medio di clienti, si può derivare la distribuzione completa del tempo di attesa in M/M/1.

Il tempo che un cliente trascorre nel sistema è distribuito esponenzialmente con tasso $\mu - \lambda$:

$$P(W > t) = e^{-(\mu - \lambda)t}.$$

Questo risultato è molto utile in applicazioni in cui non basta la media ma si vuole garantire un percentile: per esempio, assicurarsi che il 95% dei clienti aspetti meno di 2 minuti.

14.2 Oltre l'esponenziale: code M/G/1

Il modello M/G/1 ha arrivi poissoniani ma tempi di servizio con distribuzione generale, caratterizzata da media $1/\mu$ e varianza σ^2 .

La formula di Pollaczek-Khinchine (P-K) generalizza il risultato di M/M/1:

$$W_q = \frac{\lambda \left(\frac{1}{\mu^2} + \sigma^2 \right)}{2(1 - \rho)}.$$

Per $\sigma^2 = 1/\mu^2$ (servizio esponenziale) si ritrova il risultato M/M/1. Per $\sigma^2 = 0$ (servizio deterministico, modello M/D/1) si ottiene esattamente la metà del tempo di attesa di M/M/1. Questo mostra quantitativamente che la variabilità del servizio conta quanto il carico.

15. Perché questo è un buon case study per il corso

Questo progetto è particolarmente adatto a un corso di metodi computazionali per modelli stocastici per almeno cinque ragioni.

Primo, introduce i processi di birth-death e la distribuzione stazionaria come autovettore del generatore in un contesto molto concreto, dove ogni formula ha un'interpretazione immediata.

Secondo, la simulazione a eventi discreti è un metodo computazionale fondamentale, diverso dall'integrazione numerica di ODE e dalla simulazione Euler-Maruyama: è utile che gli studenti abbiano tutti e tre gli strumenti nel proprio repertorio.

Terzo, le formule di Little mostrano come un risultato di grande generalita' possa essere derivato da argomenti elementari di conservazione del flusso.

Quarto, il confronto tra $M/M/1$, $M/M/k$, $M/M/1/K$ e $M/G/1$ mostra in modo molto diretto come diverse ipotesi sul processo generino diverse fenomenologie, con implicazioni pratiche immediate.

Quinto, il tema e' fortemente interdisciplinare: le code appaiono in informatica (sistemi operativi, reti), ingegneria (produzione, logistica), sanita' (pronto soccorso, sale operatorie), economia (banche, sportelli), trasporti (semafori, porti).

16. Conclusione

La teoria delle code mostra come strumenti matematici relativamente semplici — processi di Poisson, distribuzioni esponenziali, catene di Markov a tempo continuo — permettano di rispondere a domande concrete e pratiche su sistemi reali.

Il modello $M/M/1$ e' il punto di partenza: elegante, trattabile, ricco di intuizioni. Le varianti $M/M/k$, $M/M/1/K$ e le reti di code di Jackson mostrano come lo stesso formalismo si estenda a situazioni piu' complesse.

Dal punto di vista metodologico, il progetto combina in modo naturale:

- analisi analitica di un processo stocastico a tempo continuo;
- distribuzione stazionaria come soluzione di un sistema lineare;
- formule di conservazione del flusso (legge di Little);
- simulazione a eventi discreti;
- confronto tra risultati analitici e simulati.

Il messaggio concettuale piu' importante e' forse il piu' semplice: non basta sapere il carico medio di un sistema per prevedere le code. La variabilita' conta quanto la media, e avvicinarsi alla saturazione fa crescere l'attesa in modo non lineare e spesso sorprendente.

17. Bibliografia minima

1. Kleinrock, L. (1975). Queueing Systems, Volume 1: Theory. Wiley.
2. Gross, D., Shortle, J. F., Thompson, J. M., and Harris, C. M. (2008). Fundamentals of Queueing Theory. Wiley.
3. Bertsekas, D., and Gallager, R. (1987). Data Networks. Prentice Hall.
4. Jackson, J. R. (1957). Networks of Waiting Lines. Operations Research, 5(4), 518-521.
5. Little, J. D. C. (1961). A Proof for the Queueing Formula $L = \lambda W$. Operations Research, 9(3), 383-387.

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python i principali componenti del progetto:

1. formule analitiche per M/M/1, M/M/k, M/M/1/K;
2. simulazione a eventi discreti per M/M/1;
3. estensione a M/M/1/K;
4. simulazione di una rete a due stazioni in tandem.

Il codice e' volutamente elementare:

- poche librerie;
- funzioni corte;
- cicli espliciti;
- nomi leggibili.

Non e' necessario usare `numpy` in una prima implementazione.

A.1 Librerie minime

```
import random
import math
import statistics
import matplotlib.pyplot as plt
```

A.2 Formule analitiche per M/M/1

```
def mm1_metrics(lam, mu):
    rho = lam / mu

    if rho >= 1.0:
        return None

    L = rho / (1.0 - rho)
    Lq = rho ** 2 / (1.0 - rho)
    W = 1.0 / (mu - lam)
    Wq = lam / (mu * (mu - lam))
    pi0 = 1.0 - rho

    return {
        "rho": rho,
        "L": L,
        "Lq": Lq,
        "W": W,
        "Wq": Wq,
```

```

        "pi0": pi0
    }

```

Esempio:

```

m = mm1_metrics(lam=4.0, mu=5.0)
print("rho =", m["rho"])
print("L   =", round(m["L"], 4), "clienti in media nel sistema")
print("W   =", round(m["W"], 4), "minuti in media nel sistema")
print("Wq  =", round(m["Wq"], 4), "minuti in media in attesa")

```

A.3 Distribuzione stazionaria di M/M/1

```

def mm1_stationary_distribution(rho, n_max=50):
    probs = []

    for n in range(n_max + 1):
        p_n = (1.0 - rho) * (rho ** n)
        probs.append(p_n)

    return probs

```

Grafico della distribuzione:

```

def plot_mm1_distribution(rho, n_max=30):
    probs = mm1_stationary_distribution(rho, n_max)
    ns = list(range(n_max + 1))

    plt.bar(ns, probs)
    plt.xlabel("numero di clienti nel sistema")
    plt.ylabel("probabilita'")
    plt.title(f"Distribuzione stazionaria M/M/1, rho = {rho}")
    plt.show()

```

A.4 Formule per M/M/1/K

```

def mm1k_metrics(lam, mu, K):
    rho = lam / mu

    if abs(rho - 1.0) < 1e-10:
        pi0 = 1.0 / (K + 1.0)
        probs = [pi0] * (K + 1)
    else:
        pi0 = (1.0 - rho) / (1.0 - rho ** (K + 1))
        probs = [pi0 * (rho ** n) for n in range(K + 1)]

    p_block = probs[K]
    lam_eff = lam * (1.0 - p_block)

```

```

L = sum(n * probs[n] for n in range(K + 1))

if lam_eff > 0.0:
    W = L / lam_eff
else:
    W = float("inf")

return {
    "rho": rho,
    "p_block": p_block,
    "lam_eff": lam_eff,
    "L": L,
    "W": W,
    "probs": probs
}

```

Esempio:

```

m = mmk_metrics(lam=6.0, mu=5.0, K=10)
print("Probabilita' di blocco:", round(m["p_block"], 4))
print("Tasso effettivo di arrivi:", round(m["lam_eff"], 4))
print("Numero medio nel sistema:", round(m["L"], 4))

```

A.5 Formula di Erlang C per M/M/k

```

def erlang_c(k, rho):
    a = k * rho

    sum_terms = 0.0
    factorial = 1.0

    for n in range(k):
        if n > 0:
            factorial *= n
        sum_terms += (a ** n) / factorial

    factorial_k = factorial * k
    last_term = (a ** k) / (factorial_k * (1.0 - rho))

    c = last_term / (sum_terms + last_term)
    return c

def mmk_metrics(lam, mu, k):
    rho = lam / (k * mu)

```

```

if rho >= 1.0:
    return None

C = erlang_c(k, rho)
Wq = C / (k * mu - lam)
W = Wq + 1.0 / mu
Lq = lam * Wq
L = lam * W

return {
    "rho": rho,
    "C": C,
    "Wq": Wq,
    "W": W,
    "Lq": Lq,
    "L": L
}

```

Esempio – confronto M/M/1 e M/M/2 a parita' di carico totale:

```

lam = 4.0
mu = 5.0

m1 = mm1_metrics(lam=lam, mu=mu)
m2 = mmk_metrics(lam=lam, mu=mu, k=2)

print("M/M/1 -- Wq:", round(m1["Wq"], 4))
print("M/M/2 -- Wq:", round(m2["Wq"], 4))
print("Riduzione:", round(1.0 - m2["Wq"] / m1["Wq"], 3), "frazione")

```

A.6 Generazione di variabili esponenziali

```

def exp_sample(rate):
    return random.expovariate(rate)

```

Questa funzione e' il mattone di base di tutta la simulazione.

A.7 Simulazione a eventi discreti per M/M/1

La struttura centrale e' una lista di eventi futuri ordinata per tempo.

```

def simulate_mm1(lam, mu, total_time, warmup_time=0.0):
    # stato corrente
    n = 0
    t = 0.0

    # lista degli eventi: ogni evento e' una coppia (tempo, tipo)
    # tipo: "arrival" o "departure"

```

```

events = []

# primo arrivo
t_first_arrival = exp_sample(lam)
events.append((t_first_arrival, "arrival"))

# raccolta dati (solo dopo il warmup)
n_samples = []
t_last = 0.0
time_in_state = {}

def record_state(t_now):
    dt = t_now - t_last
    if n not in time_in_state:
        time_in_state[n] = 0.0
    time_in_state[n] += dt

while events:
    # estrai il prossimo evento
    events.sort()
    t_event, event_type = events.pop(0)

    if t_event > total_time:
        record_state(total_time)
        break

    record_state(t_event)
    t_last = t_event
    t = t_event

    if event_type == "arrival":
        n += 1

        # programma il prossimo arrivo
        t_next_arrival = t + exp_sample(lam)
        events.append((t_next_arrival, "arrival"))

        # se il server era libero, inizia il servizio
        if n == 1:
            t_departure = t + exp_sample(mu)
            events.append((t_departure, "departure"))

    elif event_type == "departure":
        n -= 1

        # se ci sono ancora clienti, inizia il prossimo servizio

```

```

        if n >= 1:
            t_departure = t + exp_sample(mu)
            events.append((t_departure, "departure"))

        # campiona lo stato dopo il warmup
        if t >= warmup_time:
            n_samples.append(n)

        # calcola la distribuzione empirica e le medie
        t_total_measured = sum(time_in_state.get(k, 0.0)
                                for k in time_in_state
                                if True)

        empirical_dist = {}
        for state, time_spent in time_in_state.items():
            empirical_dist[state] = time_spent / total_time

        L_empirical = sum(state * p for state, p in empirical_dist.items())

        results = {
            "empirical_distribution": empirical_dist,
            "L_empirical": L_empirical,
            "n_samples": n_samples
        }

    return results

```

Esempio:

```

lam = 4.0
mu = 5.0

sim = simulate_mm1(lam=lam, mu=mu, total_time=10000.0, warmup_time=200.0)

analytic = mm1_metrics(lam=lam, mu=mu)

print("L analitico:", round(analytic["L"], 4))
print("L simulato: ", round(sim["L_empirical"], 4))

```

A.8 Confronto grafico tra distribuzione simulata e teorica

```

def plot_mm1_comparison(sim_results, rho, n_max=20):
    emp = sim_results["empirical_distribution"]
    theoretical = mm1_stationary_distribution(rho, n_max)

    ns = list(range(n_max + 1))
    emp_vals = [emp.get(n, 0.0) for n in ns]

```

```

plt.bar([n - 0.2 for n in ns], emp_vals, width=0.4, label="simulazione")
plt.bar([n + 0.2 for n in ns], theoretical, width=0.4, label="teorico")
plt.xlabel("n")
plt.ylabel("probabilita'")
plt.title(f"M/M/1: simulato vs teorico, rho = {rho}")
plt.legend()
plt.show()

```

A.9 Curva del tempo di attesa in funzione di rho

```

def plot_wq_vs_rho(mu=1.0, rho_max=0.99, num_points=100):
    rho_values = []
    wq_values = []

    for n in range(1, num_points + 1):
        rho = rho_max * n / num_points
        lam = rho * mu
        wq = lam / (mu * (mu - lam))
        rho_values.append(rho)
        wq_values.append(wq)

    plt.plot(rho_values, wq_values)
    plt.xlabel("rho")
    plt.ylabel("Wq (tempo medio di attesa)")
    plt.title("M/M/1: esplosione dell'attesa vicino alla saturazione")
    plt.show()

```

A.10 Simulazione di M/M/1/K

Si modifica la simulazione M/M/1 aggiungendo il rifiuto dei clienti quando il sistema e' pieno.

```

def simulate_mm1k(lam, mu, K, total_time, warmup_time=0.0):
    n = 0
    t = 0.0

    events = []
    t_first_arrival = exp_sample(lam)
    events.append((t_first_arrival, "arrival"))

    time_in_state = {}
    t_last = 0.0
    arrivals_total = 0
    arrivals_blocked = 0

```

```

def record_state(t_now):
    dt = t_now - t_last
    if n not in time_in_state:
        time_in_state[n] = 0.0
    time_in_state[n] += dt

while events:
    events.sort()
    t_event, event_type = events.pop(0)

    if t_event > total_time:
        record_state(total_time)
        break

    record_state(t_event)
    t_last = t_event
    t = t_event

    if event_type == "arrival":
        arrivals_total += 1

        if n < K:
            n += 1
            if n == 1:
                t_departure = t + exp_sample(mu)
                events.append((t_departure, "departure"))
            else:
                arrivals_blocked += 1

        t_next_arrival = t + exp_sample(lam)
        events.append((t_next_arrival, "arrival"))

    elif event_type == "departure":
        n -= 1
        if n >= 1:
            t_departure = t + exp_sample(mu)
            events.append((t_departure, "departure"))

empirical_dist = {state: time_spent / total_time
                  for state, time_spent in time_in_state.items()}

p_block_empirical = arrivals_blocked / arrivals_total if arrivals_total > 0 else 0.0

results = {
    "empirical_distribution": empirical_dist,
    "p_block_empirical": p_block_empirical,
}

```



```

        "arrivals_total": arrivals_total,
        "arrivals_blocked": arrivals_blocked
    }

    return results

```

Esempio:

```

lam = 6.0
mu = 5.0
K = 10

sim_k = simulate_mm1k(lam=lam, mu=mu, K=K, total_time=20000.0, warmup_time=500.0)
analytic_k = mm1k_metrics(lam=lam, mu=mu, K=K)

print("P_block analitico:", round(analytic_k["p_block"], 4))
print("P_block simulato: ", round(sim_k["p_block_empirical"], 4))

```

A.11 Rete a due stazioni in tandem

In una rete tandem, i clienti passano dalla stazione 1 alla stazione 2 dopo essere stati serviti.

```

def simulate_tandem_network(lam, mu1, mu2, total_time, warmup_time=0.0):
    n1 = 0
    n2 = 0
    t = 0.0

    events = []
    t_first_arrival = exp_sample(lam)
    events.append((t_first_arrival, "arrival_1"))

    n1_samples = []
    n2_samples = []
    t_last = 0.0
    time_in_state = {}

    def state():
        return (n1, n2)

    def record_state(t_now):
        dt = t_now - t_last
        s = state()
        if s not in time_in_state:
            time_in_state[s] = 0.0
        time_in_state[s] += dt

```

```

while events:
    events.sort()
    t_event, event_type = events.pop(0)

    if t_event > total_time:
        record_state(total_time)
        break

    record_state(t_event)
    t_last = t_event
    t = t_event

    if event_type == "arrival_1":
        n1 += 1
        t_next = t + exp_sample(lam)
        events.append((t_next, "arrival_1"))

        if n1 == 1:
            t_dep1 = t + exp_sample(mu1)
            events.append((t_dep1, "departure_1"))

    elif event_type == "departure_1":
        n1 -= 1
        n2 += 1

        if n1 >= 1:
            t_dep1 = t + exp_sample(mu1)
            events.append((t_dep1, "departure_1"))

        if n2 == 1:
            t_dep2 = t + exp_sample(mu2)
            events.append((t_dep2, "departure_2"))

    elif event_type == "departure_2":
        n2 -= 1

        if n2 >= 1:
            t_dep2 = t + exp_sample(mu2)
            events.append((t_dep2, "departure_2"))

    if t >= warmup_time:
        n1_samples.append(n1)
        n2_samples.append(n2)

L1 = statistics.mean(n1_samples) if n1_samples else 0.0
L2 = statistics.mean(n2_samples) if n2_samples else 0.0

```

```

results = {
    "L1_empirical": L1,
    "L2_empirical": L2,
    "n1_samples": n1_samples,
    "n2_samples": n2_samples,
    "time_in_state": time_in_state
}

return results

```

Verifica con il teorema di Jackson:

```

lam = 3.0
mu1 = 5.0
mu2 = 4.0

sim_net = simulate_tandem_network(
    lam=lam, mu1=mu1, mu2=mu2,
    total_time=30000.0, warmup_time=500.0
)

# per il teorema di Jackson, ogni stazione si comporta come M/M/1
# stazione 1: tasso arrivi = lam, stazione 2: tasso arrivi = lam (rete tandem)
m1 = mm1_metrics(lam=lam, mu=mu1)
m2 = mm1_metrics(lam=lam, mu=mu2)

print("Stazione 1 -- L analitico:", round(m1["L"], 4),
      "  L simulato:", round(sim_net["L1_empirical"], 4))
print("Stazione 2 -- L analitico:", round(m2["L"], 4),
      "  L simulato:", round(sim_net["L2_empirical"], 4))

```

A.12 Stima empirica di W dalla legge di Little

```

def little_law_check(L, lam):
    return L / lam

```

Questa funzione e' banale ma utile come controllo: verifica che $W = L/\lambda$ sia consistente con il valore simulato del tempo medio nel sistema.

A.13 Organizzazione consigliata del file

Per mantenere il codice leggibile, conviene organizzarlo in questo ordine:

1. import delle librerie;
2. funzioni analitiche:
 - mm1_metrics
 - mm1_stationary_distribution

- mm1k_metrics
 - erlang_c
 - mmk_metrics
3. funzione di campionamento:
 - exp_sample
 4. simulazione M/M/1:
 - simulate_mm1
 5. simulazione M/M/1/K:
 - simulate_mm1k
 6. rete tandem:
 - simulate_tandem_network
 7. grafici:
 - plot_mm1_distribution
 - plot_mm1_comparison
 - plot_wq_vs_rho
 8. blocco finale con esempi.

Per esempio:

```
if __name__ == "__main__":
    lam = 4.0
    mu = 5.0

    print("=== M/M/1 analitico ===")
    m = mm1_metrics(lam=lam, mu=mu)
    for k, v in m.items():
        print(f"  {k} = {round(v, 4)}")

    print("\n=== M/M/1 simulazione ===")
    sim = simulate_mm1(lam=lam, mu=mu, total_time=50000.0, warmup_time=500.0)
    print("  L simulato:", round(sim["L_empirical"], 4))

    plot_mm1_comparison(sim, rho=lam/mu, n_max=20)
    plot_wq_vs_rho(mu=mu)
```

A.14 Perché questa appendice è utile

Questa appendice ha tre funzioni didattiche.

Primo, mostra che la simulazione a eventi discreti è concettualmente molto semplice: si mantiene una lista di eventi futuri, si estrae il prossimo, si aggiorna lo stato, si generano nuovi eventi.

Secondo, il confronto diretto tra valori analitici e simulati rende molto visibile quando la teoria funziona e quanto velocemente la simulazione converge.

Terzo, la progressione $M/M/1 \rightarrow M/M/1/K \rightarrow$ rete tandem mostra come la stessa struttura di simulazione si estenda a modelli piu' complessi con modifiche minime.

A.15 Conclusione dell'appendice

La struttura proposta e' volutamente semplice. Chi conosce Python puo' implementarla quasi direttamente; chi usa altri linguaggi puo' leggerla come pseudocodice molto vicino a una traduzione operativa.

Il messaggio metodologico centrale e' che la simulazione a eventi discreti e' un complemento indispensabile all'analisi analitica: dove la teoria fornisce formule chiuse, la simulazione le verifica; dove la teoria non arriva, la simulazione continua.